



**R.D. & S.H. NATIONAL COLLEGE & S.W.A. SCIENCE
COLLEGE**

Bandra, Mumbai-400050

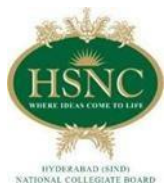
**Department of Computer Science &
Data Science**

Time Series Analysis and Forecasting

**M.Sc. Data Science Part I –Semester
II JOURNAL 2025-2026**

Course Code: - PSDSP514

Seat No. : _____



R.D. & S.H. NATIONAL COLLEGE & S.W.A. SCIENCE COLLEGE

Bandra, Mumbai-400050.



Department of Computer Science & Data Science

CERTIFICATE

This is to certify that **Mr. Molla Zamir Hossain** of MSc. Part I (Sem II) class has satisfactorily completed **10/10** Practical in the subject of **Time Series Analysis And Forecasting** as a part of M.Sc. Degree Course in Data Science during the academic year 2025– 2026.

Faculty In-Charge

Co-Ordinator
Department of Computer Science

External Examiner

INDEX

Sr. No.	Title	Date	Page No.	Sign
1	Fitting and plotting of modified exponential curve.		4-5	
2	Fitting and plotting of Gompertz curve.		6-7	
3	Fitting and plotting of logistic curve.		8-9	
4	Fitting of trend by Moving Average Method.		10-11	
5	Measurement of Seasonal indices Ratio-to-Trend method.		12-13	
6	Measurement of Seasonal indices Ratio-to-Moving Average method.		14-16	
7	Measurement of seasonal indices Link Relative method.		17-19	
8	Calculation of variance of random component by variate difference method.		20	
9	Calculation of variance of random component by variate difference method.		21	
10	Forecasting by short term forecasting methods.		22	

Practical 1

Aim: Fitting and plotting of modified exponential curve.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.optimize import curve_fit

# Modified exponential model

def mod_exp(x, a, b, c):

    return a * np.exp(b * x) + c

# Example data (replace with your dataset)

x = np.arange(1, 7)

y = 2 * np.exp(0.6 * x)

# Curve fitting

params, covariance = curve_fit(mod_exp, x, y)

a, b, c = params

print("Fitted parameters:", params)

# Smooth curve

x_fit = np.linspace(min(x), max(x), 100)

y_fit = mod_exp(x_fit, a, b, c)

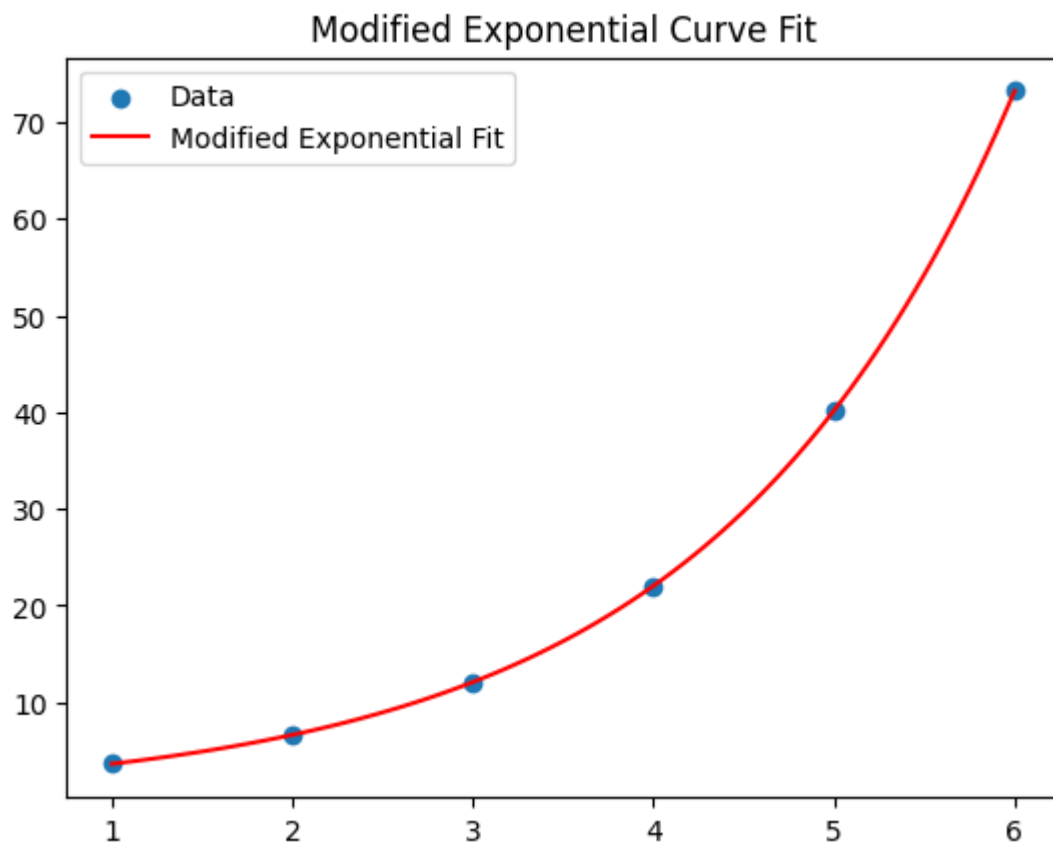
# Plot

plt.scatter(x, y, label="Data")
```

```
plt.plot(x_fit, y_fit, label="Modified Exponential Fit", color="red")  
plt.legend()  
plt.title("Modified Exponential Curve Fit")  
plt.show()
```

Output:

Fitted parameters: [1.99999994e+00 6.00000005e-01 2.07516559e-07]



Practical 2

Aim: Fitting and plotting of Gompertz curve.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.optimize import curve_fit

# Gompertz function
def gompertz(x, a, b, c):
    return a * np.exp(-b * np.exp(-c * x))

# Example data (replace with your dataset)
x = np.array([1, 2, 3, 4, 5, 6, 7])
y = np.array([1.2, 2.5, 5.1, 9.8, 15.2, 18.5, 19.2])

# Curve fitting
params, covariance = curve_fit(gompertz, x, y, maxfev=10000)

a, b, c = params
print("Fitted parameters:", params)

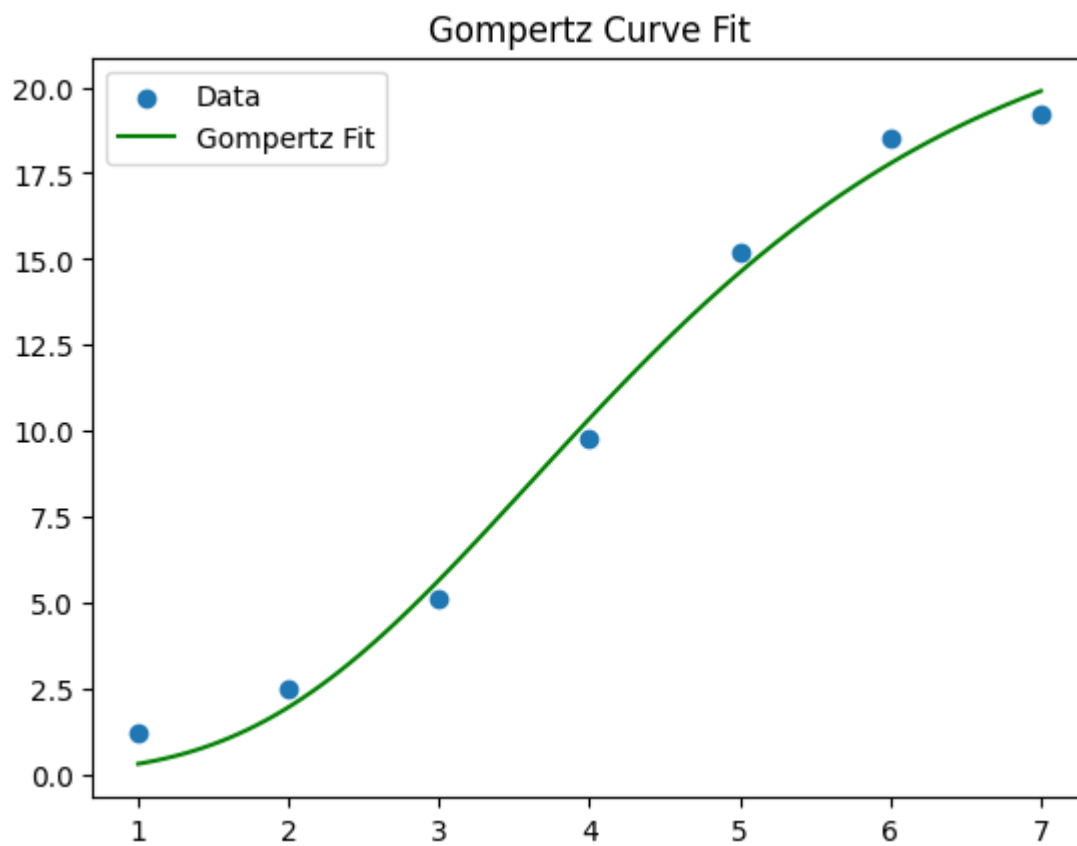
# Smooth curve
x_fit = np.linspace(min(x), max(x), 100)
y_fit = gompertz(x_fit, a, b, c)

# Plot
plt.scatter(x, y, label="Data")
```

```
plt.plot(x_fit, y_fit, label="Gompertz Fit", color="green")  
plt.legend()  
plt.title("Gompertz Curve Fit")  
plt.show()
```

Output:

Fitted parameters: [23.06682132 7.60565133 0.56266152]



Practical 3

Aim: Fitting and plotting of logistic curve.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Logistic function
def logistic(t, L, a, b):
    return L / (1 + a * np.exp(-b * t))

# Example data (replace with your dataset)
t = np.arange(1, 8)
y = np.array([5, 9, 18, 35, 55, 72, 85])

# Curve fitting
params, covariance = curve_fit(logistic, t, y, maxfev=10000)

L, a, b = params
print("Fitted parameters:", params)

# Smooth curve for plotting
t_fit = np.linspace(min(t), max(t), 100)
y_fit = logistic(t_fit, L, a, b)

# Plot
plt.plot(t, y, label="Data", linestyle='--', color="blue", alpha=0.7)
```



```
plt.plot(t_fit, y_fit, color="red", label="Logistic Fit", alpha=0.8)
```

```
plt.title("Logistic Curve Fitting")
```

```
plt.xlabel("Time (t)")
```

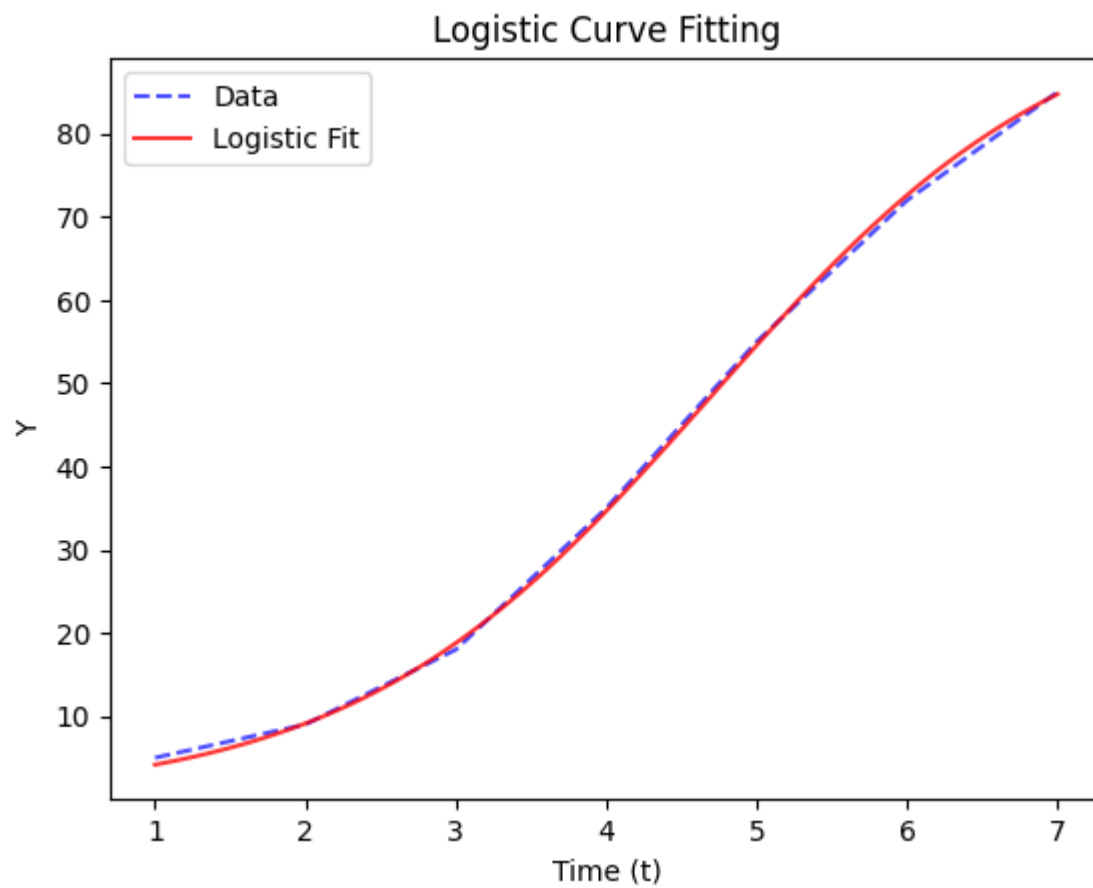
```
plt.ylabel("Y")
```

```
plt.legend()
```

```
plt.show()
```

Output:

Fitted parameters: [97.10095119 51.54184271 0.83828753]



Practical 4

Aim: Fitting of trend by Moving Average Method.

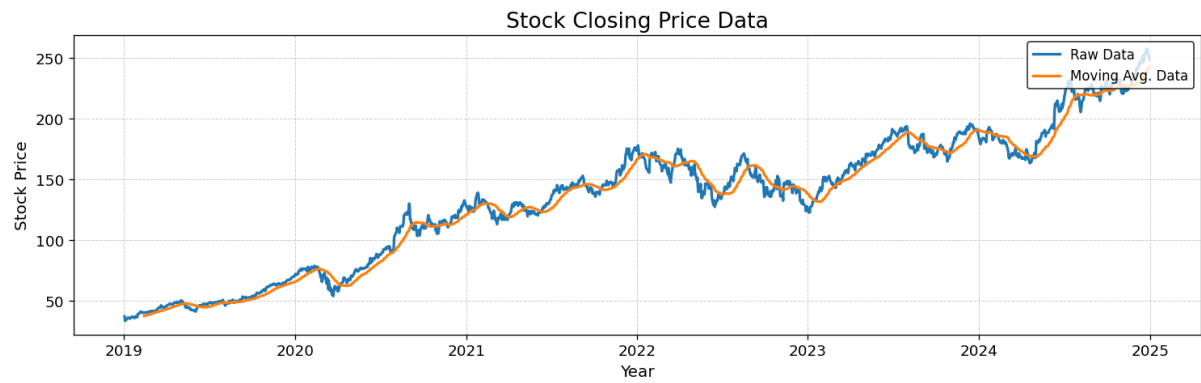
Code:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import yfinance as yf
import warnings
warnings.filterwarnings("ignore")

df = yf.download("AAPL", start="2019-01-01", end="2026-01-01")
df.sort_values(by="Date", ascending=True, inplace=True)
df.columns = df.columns.get_level_values(0)
df.columns.name = None
df.reset_index(inplace=True)

df['Moving-Avg'] = df['Close'].rolling(window=30).mean()
df['Trend'] = df['Moving-Avg'].diff()
df[25:35]

plt.plot(df['Date'], df['Close'], label='Raw Data')
plt.plot(df['Date'], df['Moving-Avg'], label='Moving Avg. Data')
plt.legend()
plt.xlabel('Year')
plt.ylabel('Stock Price')
plt.title('Stock Closing Price Data')
plt.show()
```



Visualizing Trend and Seasonal Variation

```
df['Seasonal_Variation'] = df['Close'] - df['Moving-Avg']
```

```
plt.plot(df['Date'], df['Trend'], label = 'Trend')
```

```
plt.plot(df['Date'], df['Seasonal_Variation'], label = 'Seasonal Variation')
```

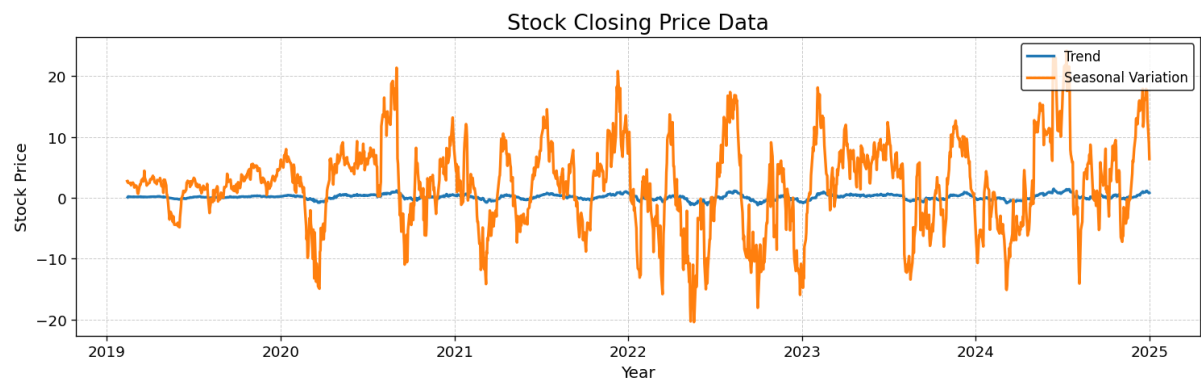
```
plt.legend()
```

```
plt.xlabel('Year')
```

```
plt.ylabel('Stock Price')
```

```
plt.title('Stock Closing Price Data')
```

```
plt.show()
```



Practical 5

Aim: Measurement of Seasonal indices Ratio-to-Trend method.

Code:

```
df['Month'] = df['Date'].dt.month
```

```
df['Year'] = df['Date'].dt.year
```

```
monthly_avg = df.groupby(['Month', 'Year'])['Close'].mean().unstack()
```

```
for i in range(2021, 2026):
```

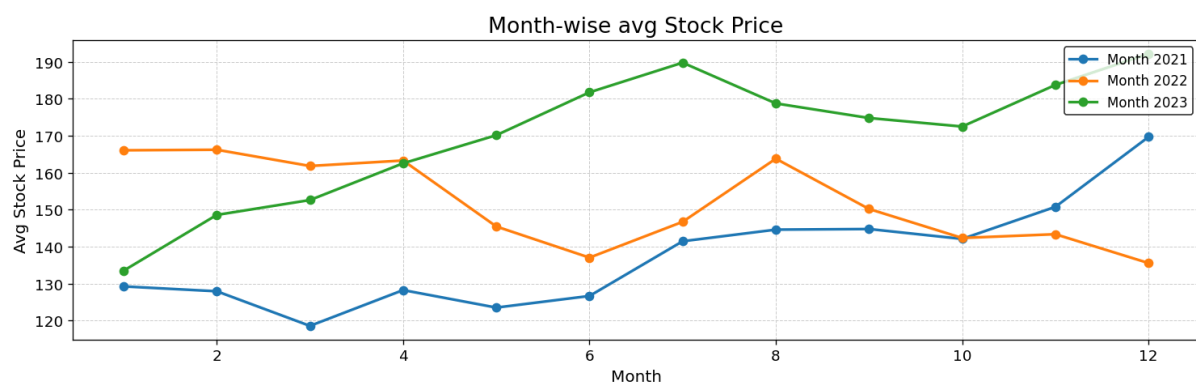
```
    plt.plot(monthly_avg.index, monthly_avg[i], label=f'Month {i}', marker='o')
```

```
    plt.legend(loc='upper left')
```

```
    plt.title('Month-wise avg Stock Price')
```

```
    plt.xlabel('Month')
```

```
    plt.ylabel('Avg Stock Price')
```



```
yearly_mean = df.groupby('Year')['Close'].mean()
```

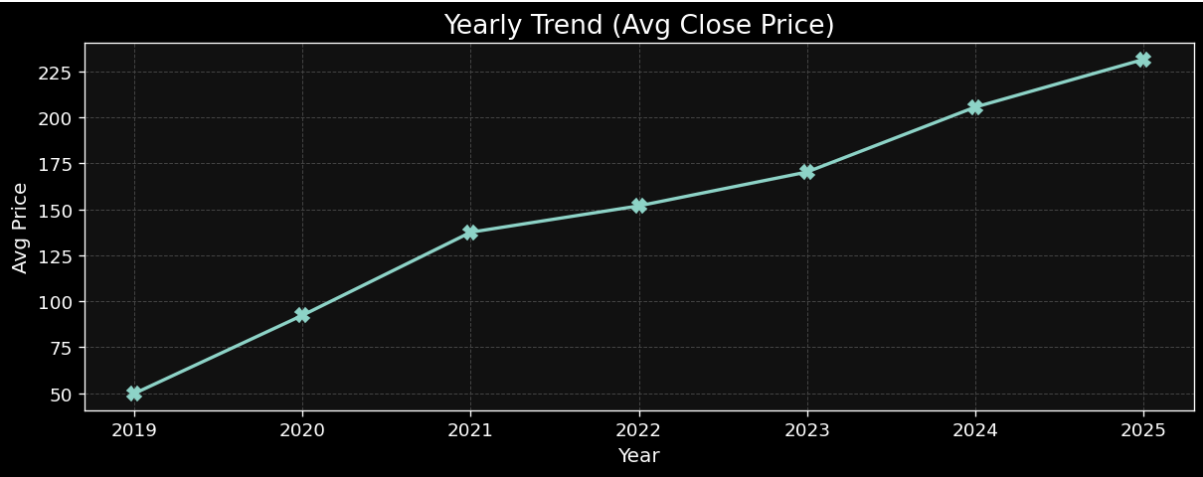
```
plt.plot(yearly_mean.index, yearly_mean.values, marker='X', markersize=8)
```

```
plt.title('Yearly Trend (Avg Close Price)')
```

```
plt.xlabel('Year')
```

```
plt.ylabel('Avg Price')
```

```
plt.show()
```



Practical 6

Aim: Measurement of Seasonal indices Ratio-to-Moving Average method.

Code: `year_cols = [2019, 2020, 2021, 2022, 2023, 2024]`

`# Monthly pivot already assumed created`

`# monthly_avg = df.groupby(['Month','Year'])['Close'].mean().unstack()`

`# 1. Monthly Mean (ONLY years)`

`monthly_avg['Monthly Mean'] = monthly_avg[year_cols].mean(axis=1)`

`# 2. CMA (smoother trend)`

`monthly_avg['CMA'] = monthly_avg['Monthly Mean'].rolling(`

`window=3, center=True, min_periods=1`

`).mean()`

`# 3. Ratio`

`monthly_avg['Ratio'] = monthly_avg['Monthly Mean'] / monthly_avg['CMA']`

`# 4. Seasonal Index`

`monthly_avg['Seasonal Index'] = (`

`monthly_avg['Ratio'] * len(monthly_avg) / monthly_avg['Ratio'].sum()`

`)`

`# 5. Deseasonalized (clean loop)`

`for year in year_cols:`

`monthly_avg[f'Deseasonalized_{year}'] = (`

`monthly_avg[year] / monthly_avg['Seasonal Index']`

`)`

`# Yearly trends`

```

for year in year_cols:
    plt.plot(monthly_avg.index, monthly_avg[year], marker='o', label=str(year))

plt.title("Monthly Average Close Price by Year")
plt.xlabel("Month")
plt.ylabel("Close Price")
plt.legend()
plt.show()

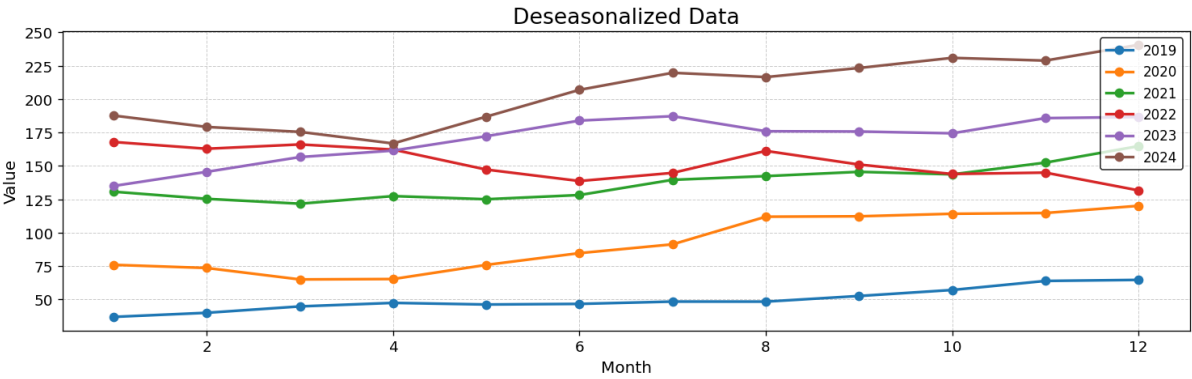
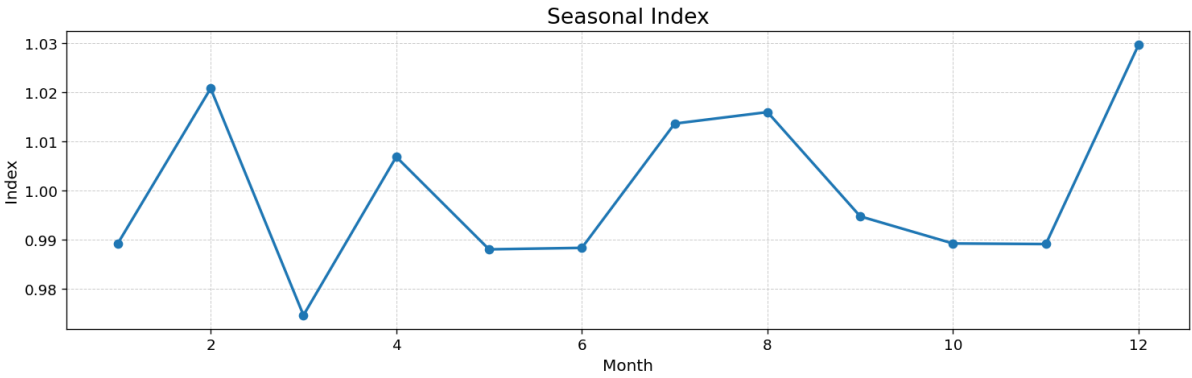
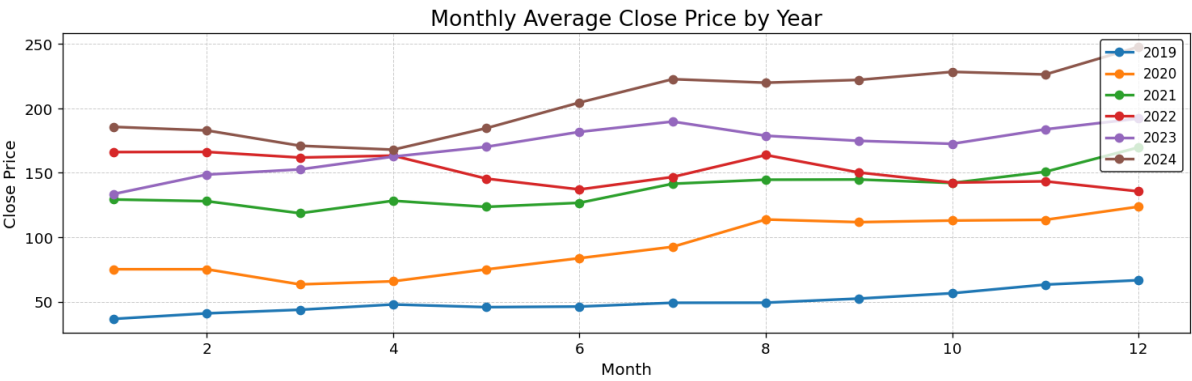
# Seasonal Index
plt.plot(monthly_avg.index, monthly_avg['Seasonal Index'], marker='o')
plt.title("Seasonal Index")
plt.xlabel("Month")
plt.ylabel("Index")
plt.show()

# Deseasonalized Data
for year in year_cols:
    plt.plot(monthly_avg.index,
             monthly_avg[f'Deseasonalized_{year}'],
             label=str(year))

plt.title("Deseasonalized Data")
plt.xlabel("Month")
plt.ylabel("Value")
plt.legend()
plt.show()

```

Output:



Practical 7

Aim: Measurement of seasonal indices Link Relative method.

Code:

```
df = monthly_avg

# Calculate the link relatives
for i in range(1, len(year_cols)):
    prev_year = year_cols[i - 1]
    curr_year = year_cols[i]

    col_name = f'Link_Relative_{curr_year}_vs_{prev_year}'
    df[col_name] = df[curr_year] / df[prev_year]

monthly_avg['Average_Link_Relative'] = monthly_avg[[
    'Link_Relative_2020_vs_2019',
    'Link_Relative_2021_vs_2020',
    'Link_Relative_2022_vs_2021',
    'Link_Relative_2023_vs_2022',
    'Link_Relative_2024_vs_2023'
]].mean(axis=1)

# seasonal index
monthly_avg['Seasonal_Index'] = (
    monthly_avg['Average_Link_Relative']
)

monthly_avg['Seasonal_Index'] /= monthly_avg['Seasonal_Index'].sum()
monthly_avg['Seasonal_Index'] *= 12

# deasonalized data
for year in year_cols:
```

```

monthly_avg[f'Deseasonalized_{year}'] = (
    monthly_avg[year] / monthly_avg['Seasonal_Index']
)

# original data
for year in year_cols:
    plt.plot(monthly_avg.index, monthly_avg[year], marker='o', label=str(year))

plt.title("Original Monthly Data")
plt.xlabel("Month")
plt.ylabel("Price")
plt.legend()
plt.show()

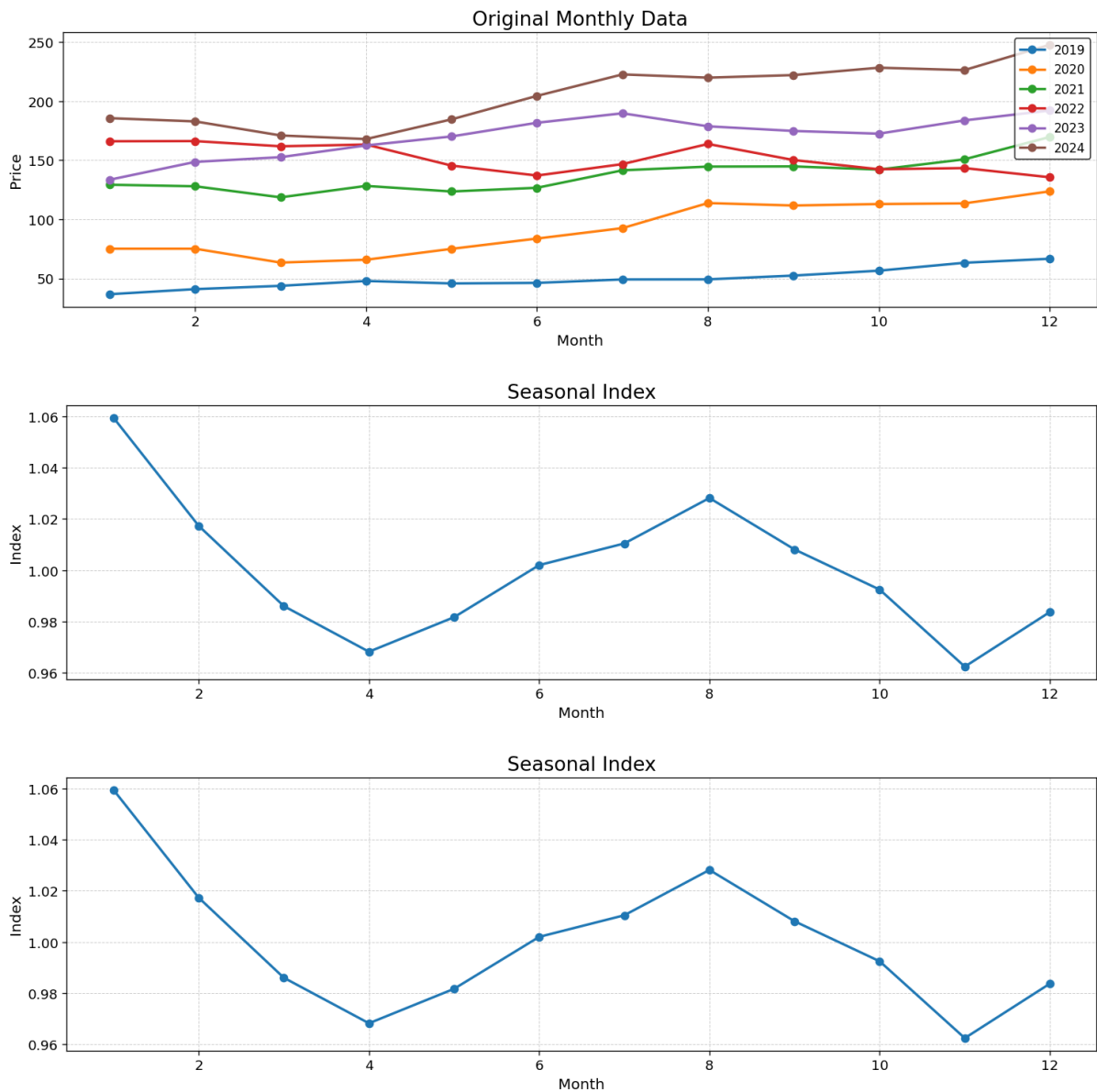
# seasonal index
plt.plot(monthly_avg.index, monthly_avg['Seasonal_Index'], marker='o')
plt.title("Seasonal Index")
plt.xlabel("Month")
plt.ylabel("Index")
plt.show()

# deseasonalized data
for year in year_cols:
    plt.plot(monthly_avg.index,
             monthly_avg[f'Deseasonalized_{year}'],
             label=str(year), marker='o')

plt.title("Deseasonalized Data")
plt.xlabel("Month")
plt.ylabel("Value")
plt.legend()
plt.show()

```

Output:



Practical 8

Aim: Calculation of variance of random component by variate difference method.

Code:

```
price_arr = df['Close']
diff = np.diff(price_arr)
mean_diff = np.mean(diff)
var_diff = np.var(diff)
var_random = var_diff / 2

print(f"Mean of Differences: {mean_diff}")
print(f"Variance of Differences: {var_diff}")
print(f"Variance of Random Components: {var_random}")

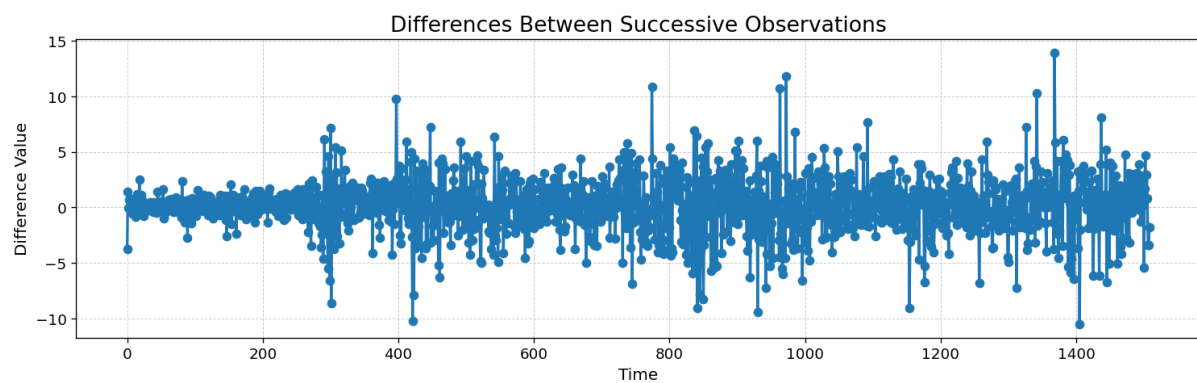
plt.plot(diff, marker='o')
plt.title('Differences Between Successive Observations')
plt.xlabel('Time')
plt.ylabel('Difference Value')
plt.grid(True)
plt.show()
```

Ouput:

Mean of Differences: 0.1400669317833235

Variance of Differences: 5.935757476658553

Variance of Random Components: 2.9678787383292766



Practical 9

Aim: Forecasting by exponential smoothing.

Code:

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

model = ExponentialSmoothing(
    df['Close'],
    trend=None,
    seasonal='add',
    seasonal_periods=365
)

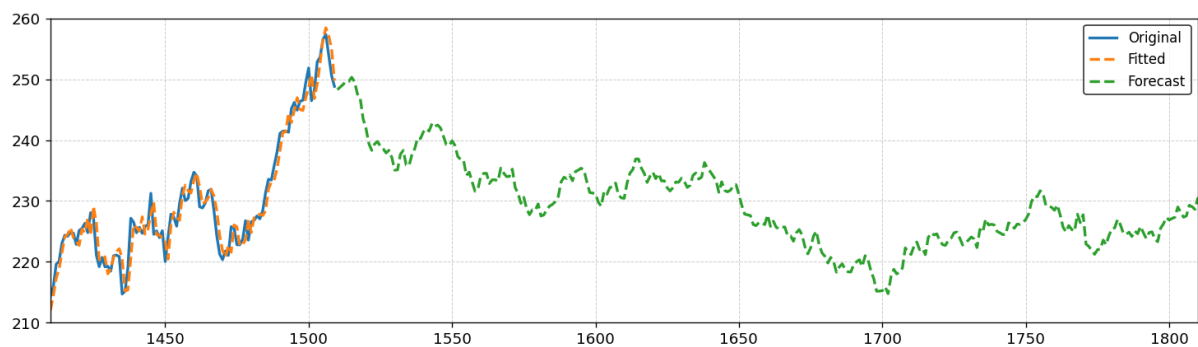
fit = model.fit()
forecast = fit.forecast(steps=365)

plt.plot(df['Close'], label='Original')
plt.plot(fit.fittedvalues, label='Fitted', linestyle='--')
plt.plot(forecast, label='Forecast', linestyle='--')

plt.xlim(len(df)-100, len(df)+300)
plt.ylim(210, 260)
plt.legend()
plt.show()

# Print forecasted values
print(forecast)
```

Ouput:



Practical 10

Aim: Forecasting by short term forecasting methods.

Code:

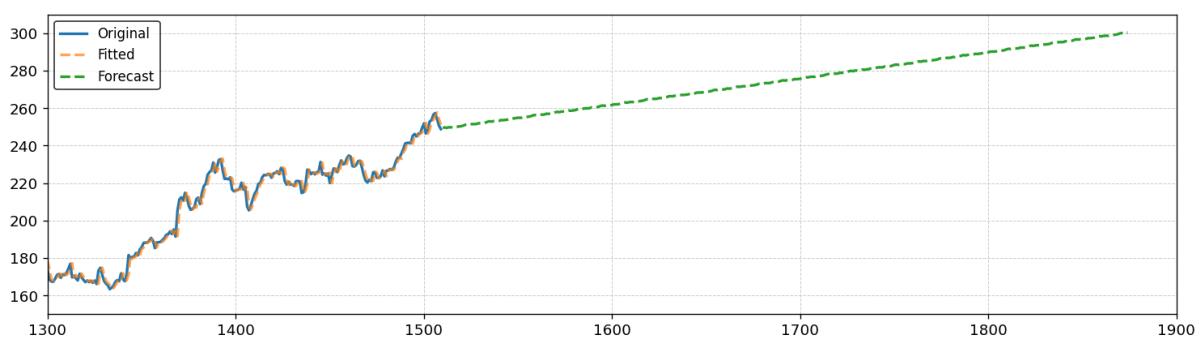
```
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(df['Close'], order=(5, 1, 1), seasonal_order=(1, 1, 1, 12)) # (p,d,q) order
fit = model.fit()
forecast = fit.forecast(steps=365)

# Plot the results
plt.plot(df['Close'], label='Original')
plt.plot(fit.fittedvalues, label='Fitted', linestyle='--', alpha=0.7)
plt.plot(forecast, label='Forecast', linestyle='--')
plt.xlim(1300, 1900)
plt.ylim(150, 310)

plt.legend(loc='upper left')
plt.show()
print(forecast)
```

Output:



1510 249.452024

1511 249.753037

1512 249.455759

1513 249.783361

Name: predicted_mean, Length: 365, dtype: float64